

**SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)**  
Department of Computer Science and Engineering

**ASSESSMENT TOOL 2 – Industry Problem Solving Task**

|                         |                                                                              |                      |                                                   |
|-------------------------|------------------------------------------------------------------------------|----------------------|---------------------------------------------------|
| <b>Course Code:</b>     | CSA06                                                                        | <b>Course Title:</b> | Design and Analysis of Algorithms                 |
| <b>CO Assessed:</b>     | <b>CO4:</b> Articulation (BL4, BL5)                                          | <b>Assessment:</b>   | Assessment Tool 2 – Industry Problem Solving Task |
| <b>Weightage:</b>       | 20% of CIA                                                                   | <b>Total Marks:</b>  | 100                                               |
| <b>CO4 Statement:</b>   | Solve optimization problems using dynamic programming and greedy strategies. |                      |                                                   |
| <b>Student Name:</b>    | GANESH M                                                                     | <b>Reg. No.:</b>     | 192524034                                         |
| <b>Section / Batch:</b> | SLOT B                                                                       | <b>Date:</b>         | 24.08.2026                                        |

**Instructions to Students**

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

| Question Type                        | Focus Area                                                                             | Questions |
|--------------------------------------|----------------------------------------------------------------------------------------|-----------|
| <b>Industry Problem Solving Task</b> | Focus on Solving optimization problems using dynamic programming and greedy strategies | Q1        |

**SECTION A – Industry Problem Solving Task**

## Code of Conduct

I GANESH M (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: \_\_\_\_\_

## RUBRICS FOR CALCULATION

### Industry Problem Solving Task

| Criteria                            | Weightage | Level 4 – Exemplary                                                          | Level 3 – Proficient                                 | Level 2 – Developing                           | Level 1 – Beginning                               |
|-------------------------------------|-----------|------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|---------------------------------------------------|
| Understanding of Industry Problem   | 20%       | Comprehensive understanding of real-world problem and constraints            | Good understanding with minor gaps in requirements   | Basic understanding; some requirements unclear | Poor understanding; misinterprets problem context |
| Application of Engineering Concepts | 25%       | Applies advanced and relevant concepts effectively to solve the problem      | Applies appropriate concepts with minor errors       | Limited application of concepts                | Incorrect or no application of concepts           |
| Solution Design                     | 25%       | Innovative, practical, and feasible solution aligned with industry standards | Logical and feasible solution with minor gaps        | Basic solution with limited feasibility        | Unclear or impractical solution                   |
| Use of Tools                        | 15%       | Effective use of modern tools, technologies, and real data                   | Appropriate use of tools with correct implementation | Limited or inconsistent use of tools           | Minimal or incorrect use of tools                 |
| Analysis                            | 10%       | Thorough analysis with validated results and performance evaluation          | Good analysis with reasonable validation             | Basic analysis with limited validation         | Poor or no analysis                               |

|                 |    |                                                                  |                                     |                                    |                                     |
|-----------------|----|------------------------------------------------------------------|-------------------------------------|------------------------------------|-------------------------------------|
| Professionalism | 5% | Highly professional, well-documented, and clearly presented work | Good documentation and presentation | Basic documentation with some gaps | Poor documentation and presentation |
|-----------------|----|------------------------------------------------------------------|-------------------------------------|------------------------------------|-------------------------------------|

### Marks Evaluation:

| Criteria                            | Wt. | Level 4 – Exemplary | Level 3 – Proficient | Level 2 – Developing | Level 1 – Beginning |
|-------------------------------------|-----|---------------------|----------------------|----------------------|---------------------|
| Understanding of Industry Problem   | 20% |                     |                      |                      |                     |
| Application of Engineering Concepts | 25% |                     |                      |                      |                     |
| Solution Design                     | 25% |                     |                      |                      |                     |
| Use of Tools                        | 15% |                     |                      |                      |                     |
| Analysis                            | 10% |                     |                      |                      |                     |
| Professionalism                     | 5%  |                     |                      |                      |                     |
| <b>Total Marks (100):</b>           |     |                     |                      |                      |                     |

| Faculty In-charge | Course Coordinator | Head of Department |
|-------------------|--------------------|--------------------|
| Name: _____       | Name: _____        | Name: _____        |
| Signature: _____  | Signature: _____   | Signature: _____   |
| Date: _____       | Date: _____        | Date: _____        |

## **ASSESSMENT TOOL 2 – Industry Problem Solving Task**

**NAME: GANESH M**

**REG.NO: 192524034**

**COURSE CODE: CSA06**

**COURSE NAME: DESIGN AND ANALYSIS OF ALGORITHMS**

**DATE: 24.08.2026**

### **Q13. Smart Warehouse Storage Optimization (Bin Packing)**

A warehouse must efficiently allocate packages of varying sizes into storage units with fixed capacity. Analyze how this problem can be modeled using Bin Packing algorithms. Identify constraints such as space utilization and item distribution. Design a feasible solution using greedy heuristics like First-Fit or Best-Fit. Discuss trade-offs between optimality and computational efficiency. Evaluate algorithm performance in large-scale warehouses. Analyze time complexity and justify solution effectiveness in logistics applications.

#### **1. Problem Statement**

A warehouse receives packages of different sizes or volumes and must assign them to storage units (bins) whose capacity is fixed. The objective is to use as few storage units as possible while ensuring that no storage unit exceeds its capacity.

This is a classical Bin Packing optimization problem. In the warehouse interpretation, each package is an item and each storage unit is a bin. The algorithm must decide where every package should be placed while minimizing the number of bins opened.

- Minimize the number of storage units used.
- Never exceed the capacity of any storage unit.
- Maintain high space utilization.
- Handle packages with highly variable sizes.
- Produce decisions fast enough for warehouse operations.
- Scale from small batches to large package waves.

## 2. Real-Time Industry Scenario

Modern warehouses receive continuous streams of parcels from e-commerce orders, manufacturing dispatches, retail replenishment and third-party logistics operations. A storage allocation system must decide where each package should be placed without wasting capacity or creating excessive fragmentation.

For example, assume a warehouse uses storage units with capacity 100 units. A batch contains packages sized 60, 40, 30, 70, 20, 50 and 10. A naive allocation order may open additional bins unnecessarily. A decreasing-size strategy places large packages first and then fills the remaining gaps with smaller packages.

### Operational Context

| Warehouse factor             | Effect on packing                                  |
|------------------------------|----------------------------------------------------|
| Variable package sizes       | Creates fragmentation and uneven residual capacity |
| Fixed bin capacity           | Creates a hard feasibility constraint              |
| High package arrival rate    | Requires low decision latency                      |
| Frequent retrieval           | Packing should not make items difficult to access  |
| Different package priorities | Urgent items may require accessible locations      |
| Physical dimensions          | Volume alone may not represent physical fit        |
| Weight limits                | A bin can have both volume and weight constraints  |

### Why the problem matters

Poor storage allocation increases the number of occupied units, reduces available warehouse capacity and can increase handling. Efficient packing can postpone the need for additional storage space and improve the utilization of existing capacity.

### 3. Bin Packing Model

Let there be  $n$  packages with sizes  $s_1, s_2, \dots, s_n$  and storage units with identical capacity  $C$ . Let  $x_{ij}$  indicate whether package  $i$  is assigned to storage unit  $j$ , and let  $y_j$  indicate whether storage unit  $j$  is used.

#### Objective

Minimize:  $\sum y_j$

#### Capacity Constraint

For every storage unit  $j$ :

$$\sum (s_i \times x_{ij}) \leq C \times y_j$$

#### Assignment Constraint

Every package must be assigned exactly once:

$$\sum x_{ij} = 1 \quad \text{for every package } i$$

#### Binary Variables

$$x_{ij} \in \{0,1\}, \quad y_j \in \{0,1\}$$

#### Interpretation

| Mathematical term | Warehouse meaning                   |
|-------------------|-------------------------------------|
| $s_i$             | Size/volume of package $i$          |
| $C$               | Capacity of a storage unit          |
| $x_{ij}$          | Package $i$ is assigned to unit $j$ |
| $y_j$             | Storage unit $j$ is opened/used     |
| $\sum y_j$        | Total storage units required        |

The exact model is computationally difficult for large instances. Therefore, practical warehouses commonly use fast heuristics or approximation methods when a decision must be made quickly.

## 4. Warehouse Constraints

A useful warehouse model must capture more than the simple capacity equation. The following constraints influence the quality and feasibility of the allocation.

| Constraint        | Description                                         | Impact                                             |
|-------------------|-----------------------------------------------------|----------------------------------------------------|
| Capacity          | Total package size cannot exceed unit capacity      | Hard feasibility requirement                       |
| Space utilization | Unused space should be minimized                    | Controls storage efficiency                        |
| Item distribution | Packages may have very different sizes              | Creates fragmentation                              |
| Weight            | Storage racks/units may have weight limits          | May reject an otherwise volume-feasible assignment |
| Dimensions        | Length, width and height can matter                 | Creates multidimensional packing                   |
| Compatibility     | Hazardous or incompatible items may need separation | Adds conflict constraints                          |
| Temperature/zone  | Some products require specific zones                | Restricts eligible units                           |
| Accessibility     | Frequently picked items should remain reachable     | Can conflict with maximum packing density          |
| Dynamic arrivals  | Future packages are unknown                         | Favors online heuristics                           |

### Key Design Principle

The mathematically densest packing is not always the operationally best packing. A warehouse should consider travel, rehandling and accessibility in addition to capacity utilization.

## 5. Bin Packing Algorithms Considered

| Algorithm                  | Strategy                                                  | Strength                        | Limitation                                |
|----------------------------|-----------------------------------------------------------|---------------------------------|-------------------------------------------|
| First Fit (FF)             | Place item in the first bin with enough space             | Very simple and fast            | Can leave fragmentation                   |
| Best Fit (BF)              | Place item in the feasible bin with least remaining space | Reduces local leftover gaps     | Requires more searching                   |
| First Fit Decreasing (FFD) | Sort items descending, then apply First Fit               | Strong practical baseline       | Sorting adds overhead; not always optimal |
| Best Fit Decreasing (BFD)  | Sort descending, then apply Best Fit                      | Often produces compact packings | More placement work                       |

### Selected Algorithm

The Python implementation in this report uses First Fit Decreasing (FFD). FFD is a greedy heuristic: it makes a locally feasible decision for each package after sorting packages from largest to smallest.

### Why FFD was selected

- Large packages are placed before small packages consume residual capacity.
- It is easy to implement and explain in an academic setting.
- It provides fast approximate solutions without exhaustive search.
- It is suitable for batch-based warehouse allocation.
- Its result can be evaluated against a simple capacity lower bound.



## 6. First Fit Decreasing – Working Principle

FFD has two stages. First, package sizes are sorted in decreasing order. Second, each package is scanned against the currently opened bins from the beginning. The package is inserted into the first bin that has sufficient remaining capacity. If no bin can accommodate it, a new bin is opened.

### Step-by-Step Example

| Step | Package | Bin state after placement            |
|------|---------|--------------------------------------|
| 1    | 70      | Bin 1 = 70/100                       |
| 2    | 60      | Bin 1 = 70; Bin 2 = 60               |
| 3    | 50      | Bin 1 = 70; Bin 2 = 60; Bin 3 = 50   |
| 4    | 40      | Bin 1 = 70; Bin 2 = 60; Bin 3 = 90   |
| 5    | 30      | Bin 1 = 100; Bin 2 = 60; Bin 3 = 90  |
| 6    | 20      | Bin 1 = 100; Bin 2 = 80; Bin 3 = 90  |
| 7    | 10      | Bin 1 = 100; Bin 2 = 90; Bin 3 = 100 |

Final packing: Bin 1 = 70 + 30 = 100; Bin 2 = 60 + 20 + 10 = 90; Bin 3 = 50 + 40 = 90.

Total package size = 280. Three bins provide total capacity 300. Therefore, unused capacity is 20 and utilization is  $280/300 \times 100 = 93.33\%$ .

### Theoretical Lower Bound

A quick lower bound is  $\text{ceil}(\text{total package size} / \text{bin capacity}) = \text{ceil}(280/100) = 3$ . Since FFD also uses 3 bins, this sample reaches the lower bound.

## 7. Proposed Algorithm

### Input

- Number of packages  $n$
- Package sizes
- Storage unit capacity  $C$

### Processing

1. Read the number of packages and storage capacity.
2. Read all package sizes.
3. Sort the packages from largest to smallest.
4. Create an empty list of storage bins.
5. For each package, scan the bins in order.
6. If a bin has enough remaining capacity, place the package there.
7. If no existing bin can hold the package, open a new bin.
8. Calculate total used space, number of bins, unused space and utilization.
9. Calculate the theoretical lower bound for comparison.
10. Display the packing performance.

### Algorithm Flow

START → Input packages and capacity → Sort descending → Select next package → Existing bin can fit? → YES: place package → NO: open new bin → More packages? → YES: repeat → NO: calculate metrics → Display results → END

### Pseudocode

```
START
Input n, capacity, package sizes
Sort package sizes in decreasing order
bins = empty list

For each package:
    placed = False
    For each bin:
        If remaining capacity >= package:
            place package
            reduce remaining capacity
            placed = True
            break
    If not placed:
        open a new bin

Calculate:
    total size
    number of bins
    lower bound
    unused space
    utilization

Display results
END
```

## 8. Python Implementation

The following is the exact Python program represented in the submitted execution screenshot. The program implements First Fit Decreasing and calculates the requested warehouse performance measures.

```
import math

# Input
n = int(input("Enter number of packages: "))
capacity = int(input("Enter storage unit capacity: "))
packages = list(map(int, input("Enter package sizes: ").split()))

# Sort packages from largest to smallest
packages.sort(reverse=True)

# Store remaining capacity of each bin
bins = []

# First Fit Decreasing
for package in packages:

    placed = False

    for i in range(len(bins)):

        if bins[i] >= package:
            bins[i] -= package
            placed = True
            break

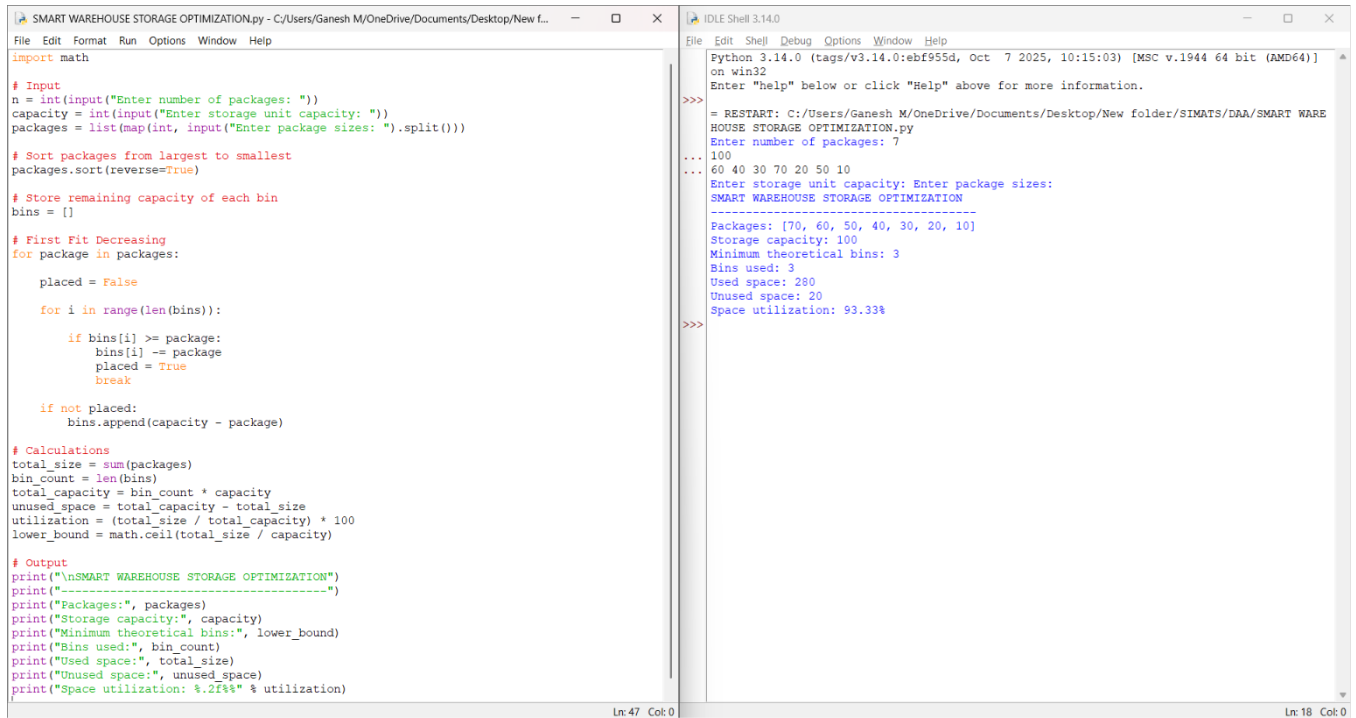
    if not placed:
        bins.append(capacity - package)

# Calculations
total_size = sum(packages)
bin_count = len(bins)
total_capacity = bin_count * capacity
unused_space = total_capacity - total_size
utilization = (total_size / total_capacity) * 100
lower_bound = math.ceil(total_size / capacity)

# Output
print("\nSMART WAREHOUSE STORAGE OPTIMIZATION")
print("-----")
print("Packages:", packages)
print("Storage capacity:", capacity)
print("Minimum theoretical bins:", lower_bound)
print("Bins used:", bin_count)
print("Used space:", total_size)
print("Unused space:", unused_space)
print("Space utilization: %.2f%%" % utilization)
```

## 9. Program Execution and Validated Result

Figure 1 shows the Python implementation and its execution in IDLE. The screenshot is included as evidence that the algorithm was executed using the sample input and produced the stated output.



```
SMART WAREHOUSE STORAGE OPTIMIZATION.py - C:/Users/Ganesh M/OneDrive/Documents/Desktop/New f...
File Edit Format Run Options Window Help

import math

# Input
n = int(input("Enter number of packages: "))
capacity = int(input("Enter storage unit capacity: "))
packages = list(map(int, input("Enter package sizes: ").split()))

# Sort packages from largest to smallest
packages.sort(reverse=True)

# Store remaining capacity of each bin
bins = []

# First Fit Decreasing
for package in packages:
    placed = False
    for i in range(len(bins)):
        if bins[i] >= package:
            bins[i] -= package
            placed = True
            break
    if not placed:
        bins.append(capacity - package)

# Calculations
total_size = sum(packages)
bin_count = len(bins)
total_capacity = bin_count * capacity
unused_space = total_capacity - total_size
utilization = (total_size / total_capacity) * 100
lower_bound = math.ceil(total_size / capacity)

# Output
print("\nSMART WAREHOUSE STORAGE OPTIMIZATION")
print("-----")
print("Packages:", packages)
print("Storage capacity:", capacity)
print("Minimum theoretical bins:", lower_bound)
print("Bins used:", bin_count)
print("Used space:", total_size)
print("Unused space:", unused_space)
print("Space utilization: %.2f%%" % utilization)
```

```
IDLE Shell 3.14.0
File Edit Shell Debug Options Window Help

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)]
on win32
Enter "help" below or click "Help" above for more information.

>>> = RESTART: C:/Users/Ganesh M/OneDrive/Documents/Desktop/New folder/SIMATS/DAA/SMART WAREHOUSE STORAGE OPTIMIZATION.py
Enter number of packages: 7
... 100
... 60 40 30 70 20 50 10
Enter storage unit capacity: Enter package sizes:
SMART WAREHOUSE STORAGE OPTIMIZATION
-----
Packages: [70, 60, 50, 40, 30, 20, 10]
Storage capacity: 100
Minimum theoretical bins: 3
Bins used: 3
Used space: 280
Unused space: 20
Space utilization: 93.33%

>>>
```

Figure 1. Python implementation and execution output for Smart Warehouse Storage Optimization using First Fit Decreasing (FFD).

### Sample Input

```
7
100
60 40 30 70 20 50 10
```

### Observed Output

```
Packages: [70, 60, 50, 40, 30, 20, 10]
Storage capacity: 100
Minimum theoretical bins: 3
Bins used: 3
Used space: 280
Unused space: 20
Space utilization: 93.33%
```

## 10. Algorithm and Code Explanation

| Code component                               | Algorithmic role               | Warehouse interpretation                    |
|----------------------------------------------|--------------------------------|---------------------------------------------|
| <code>packages.sort(reverse=True)</code>     | Decreasing-order preprocessing | Largest packages are considered first       |
| <code>bins = []</code>                       | Initialize solution            | No storage unit has been opened yet         |
| <code>for package in packages</code>         | Process every item             | Allocate every incoming package             |
| <code>for i in range(len(bins))</code>       | Search existing bins           | Check storage units in order                |
| <code>if bins[i] &gt;= package</code>        | Feasibility test               | Package fits without exceeding capacity     |
| <code>bins[i] -= package</code>              | Update state                   | Remaining free space is reduced             |
| <code>if not placed</code>                   | Failure to fit                 | No current unit can accommodate the package |
| <code>bins.append(capacity - package)</code> | Open new bin                   | Create another storage unit                 |
| <code>sum(packages)</code>                   | Total demand                   | Total package space required                |
| <code>len(bins)</code>                       | Solution size                  | Number of storage units used                |
| <code>math.ceil(total_size/capacity)</code>  | Lower bound                    | Minimum possible bins by capacity alone     |
| utilization calculation                      | Efficiency metric              | Percentage of used storage capacity         |

### Why the Code is Greedy

The algorithm does not reconsider every previous placement to search for a globally optimal arrangement. Once a package is placed, the decision is retained. This local decision-making is what makes the method computationally attractive but prevents it from guaranteeing the global optimum.

### Important distinction

The overall optimization problem is Bin Packing. The specific algorithm implemented in the code is First Fit Decreasing. First Fit and Best Fit are alternative greedy heuristics discussed for comparison; they are not simultaneously executed by the submitted program.

## 11. Performance Evaluation

The program evaluates solution quality using four measures: number of bins, theoretical lower bound, unused capacity and space utilization. These measures are appropriate for the simplified one-dimensional warehouse model.

| Metric                   | Formula / Meaning      | Result for sample |
|--------------------------|------------------------|-------------------|
| Total package size       | $\Sigma$ package sizes | 280               |
| Bin capacity             | C                      | 100               |
| Lower bound              | $\text{ceil}(280/100)$ | 3                 |
| Bins used                | Number of opened bins  | 3                 |
| Total available capacity | $3 \times 100$         | 300               |
| Unused space             | $300 - 280$            | 20                |
| Space utilization        | $280/300 \times 100$   | 93.33%            |
| Gap above lower bound    | $(3-3)/3 \times 100$   | 0%                |

### Interpretation

The sample is efficient because the algorithm uses exactly the simple capacity lower bound of three bins. The resulting utilization of 93.33% means that only 20 capacity units remain unused across the three storage units.

### Validation

The validation checks are: no bin receives more than 100 units; every input package is processed; the total used space equals the sum of all package sizes; and the calculated utilization is consistent with the number of bins used.

## 12. Time and Space Complexity

### Time Complexity

Sorting the  $n$  packages requires  $O(n \log n)$ . The basic First Fit placement uses a scan through the currently opened bins for each package. In the straightforward list implementation, the worst-case placement work is  $O(n^2)$ . Therefore, the overall worst-case complexity of the submitted implementation is  $O(n^2)$ , with sorting  $O(n \log n)$  dominated by the placement loop.

### Space Complexity

The bins list stores at most  $O(n)$  remaining-capacity values, while the input package list also stores  $O(n)$  values. Therefore, auxiliary storage is  $O(n)$ .

| Component                                  | Complexity          |
|--------------------------------------------|---------------------|
| Input storage                              | $O(n)$              |
| Decreasing sort                            | $O(n \log n)$       |
| Bin placement – simple list implementation | $O(n^2)$ worst case |
| Bin list storage                           | $O(n)$              |
| Overall time – submitted implementation    | $O(n^2)$ worst case |
| Overall auxiliary space                    | $O(n)$              |

### Why this is acceptable

For small and moderate warehouse batches,  $O(n^2)$  simple code can be acceptable because the implementation is easy to audit and the actual number of bins is often much smaller than  $n$ . For very large package streams, indexed or balanced data structures can reduce placement overhead.

## 13. Scalability for Large-Scale Warehouses

A large warehouse may process thousands or millions of packages. The same FFD concept can be retained, but the implementation architecture must change to maintain predictable latency.

| Scale        | Example workload      | Recommended approach                                               |
|--------------|-----------------------|--------------------------------------------------------------------|
| Small        | 10–100 packages       | Direct FFD list implementation                                     |
| Medium       | 1,000–10,000 packages | Batch FFD with efficient bin search                                |
| Large        | 100,000+ packages     | Indexed residual-capacity structures and parallel batch processing |
| Very dynamic | Continuous arrivals   | Online FF/BF-style allocation with periodic FFD re-optimization    |

### Scalability Strategy

- Process packages in batches instead of repeatedly re-sorting the entire warehouse.
- Use efficient residual-capacity indexing to avoid scanning every bin.
- Separate high-priority packages from ordinary batches.
- Freeze assignments that are already physically executed.
- Use monitoring to detect fragmentation and trigger re-optimization.
- Use exact optimization only for small or high-value exceptional batches.

### Important limitation

The submitted Python code is an academic reference implementation. It demonstrates the algorithm correctly but should not be treated as a production Warehouse Management System. Production software would require database integration, concurrency control, physical dimensions, safety constraints, barcode/RFID integration and real-time inventory state.



## 14. Optimality versus Computational Efficiency

Bin Packing is computationally difficult in its general form. An exact algorithm can search for a globally minimum number of bins, but the search space grows rapidly. Greedy heuristics intentionally sacrifice guaranteed optimality to produce fast decisions.

| Approach             | Solution quality          | Computation                       | Warehouse use                             |
|----------------------|---------------------------|-----------------------------------|-------------------------------------------|
| Exact optimization   | Potentially optimal       | High / may become exponential     | Small critical batches                    |
| First Fit            | Heuristic                 | Low                               | Fast online placement                     |
| Best Fit             | Heuristic                 | Moderate in simple implementation | Tight local packing                       |
| First Fit Decreasing | Strong heuristic baseline | Sorting + placement               | Batch storage planning                    |
| Best Fit Decreasing  | Strong heuristic          | More placement work               | Batch planning when tight packing matters |

### Decision Rule

A practical warehouse should not optimize only for the number of bins. If saving one storage unit causes significant rehandling or additional picker travel, the supposed optimization can increase total logistics cost.

### Recommended strategy

Use FFD as the main batch heuristic, use FF/BF for rapid online decisions, and reserve exact optimization for small exceptional batches where the value of improved packing justifies the computational cost.

## 15. Effectiveness in Logistics Applications

### E-commerce Fulfilment

Package waves can be grouped and allocated to storage units before put-away. FFD can reduce fragmentation when package sizes vary widely.

### Manufacturing Warehouses

Components and finished goods can be allocated to fixed-capacity storage areas, subject to additional weight and compatibility constraints.

### Third-Party Logistics

A 3PL facility may handle packages from multiple customers. Efficient packing can improve the use of limited storage capacity and delay expansion.

### Retail Distribution

Replenishment packages can be allocated in batches while preserving space for future arrivals.

### Recommended KPIs

| KPI                            | Purpose                                          |
|--------------------------------|--------------------------------------------------|
| Storage utilization (%)        | Measure capacity efficiency                      |
| Bins opened per 1,000 packages | Measure storage demand                           |
| Placement latency              | Measure algorithm responsiveness                 |
| Rehandling events              | Measure operational side effects                 |
| Picker travel distance         | Measure accessibility impact                     |
| Capacity violations            | Safety/feasibility KPI; target should be zero    |
| SLA impact                     | Ensure optimization does not delay urgent orders |

## 16. Limitations and Failure Modes

| Limitation             | Potential problem                               | Required improvement               |
|------------------------|-------------------------------------------------|------------------------------------|
| One-dimensional size   | Real packages have length, width and height     | Use multidimensional packing       |
| Weight ignored         | A physically fitting bin may exceed rack weight | Add weight constraints             |
| Static batch           | Future arrivals are unknown                     | Use rolling/online optimization    |
| No accessibility model | Dense packing can increase retrieval time       | Add travel/retrieval cost          |
| No compatibility rules | Unsafe co-storage could occur                   | Add conflict matrix                |
| Greedy decisions       | No global optimality guarantee                  | Use exact optimization selectively |
| Simple list search     | Runtime can grow quadratically                  | Use indexed residual capacity      |

## 17. Proposed Advanced Hybrid Extension

For an industrial system, the FFD implementation can be extended into a hybrid warehouse controller. The controller uses fast greedy placement for normal flows and invokes stronger optimization only when the expected operational benefit is high.

11. Receive package size, weight, priority and storage requirements.
12. Validate physical and policy constraints.
13. Use FFD for batch allocation.
14. Measure utilization and fragmentation.
15. If the result is below the operational threshold, accept it.
16. If the batch is small and savings justify computation, invoke an exact/constraint optimizer.
17. Freeze executed placements to avoid unnecessary rehandling.
18. Monitor warehouse KPIs and periodically retune thresholds.

### Proposed Objective

Minimize Total Logistics Cost = Storage Cost + Rehandling Cost + Travel Cost + Delay Penalty, subject to capacity, compatibility and operational constraints.

## 18. Expected Outcome

The proposed solution should allocate all packages without exceeding storage capacity while minimizing the number of storage units and maintaining high utilization. For the demonstrated sample, the FFD implementation uses three bins, which equals the simple theoretical lower bound, and achieves 93.33% utilization.

At large scale, the same strategy remains useful as a baseline, but production deployment should combine greedy packing with indexed data structures, batching, operational constraints and selective stronger optimization.

## 19. Conclusion

Smart Warehouse Storage Optimization can be effectively modeled as the Bin Packing problem. The warehouse is represented by fixed-capacity bins and packages are represented by items with varying sizes. The central optimization objective is to minimize the number of storage units while satisfying capacity constraints.

The implemented First Fit Decreasing algorithm provides a practical greedy solution. By sorting packages from largest to smallest and placing each package into the first feasible bin, the method reduces fragmentation while keeping computational requirements manageable. The submitted Python execution demonstrates a valid result of three bins for the sample workload, matching the simple capacity lower bound and producing 93.33% space utilization.

The major trade-off is that FFD is not guaranteed to find the global optimum for every input. Nevertheless, its speed, simplicity and predictable behavior make it useful for warehouse batch planning. For large industrial systems, FFD should be combined with constraint validation, efficient data structures, real-time inventory information and selective exact optimization.

## References / Conceptual Sources

1. Johnson, D. S., Fast algorithms for bin packing, Journal of Computer and System Sciences, 1974.
2. Martello, S. and Toth, P., Knapsack Problems: Algorithms and Computer Implementations, Wiley.
3. Christensen, K. I., Khan, A., Pokutta, S. and Tetali, P., Approximation and online algorithms for multidimensional bin packing: A survey, Computer Science Review.
4. Standard Bin Packing literature covering First Fit, Best Fit, First Fit Decreasing and approximation analysis.
5. Course assessment reference: SIMATS Engineering, Assessment Tool 2 – Industry Problem Solving Task, CSA06 Design and Analysis of Algorithms.